CEIMM

CENTRO DE ELECTRÓNICA INDUSTRIAL Y SISTEMAS MULTIMODALES

Lab 3: Contiki and Wireless Communication with Cookies

Gabriel Mujica
Jorge Portilla

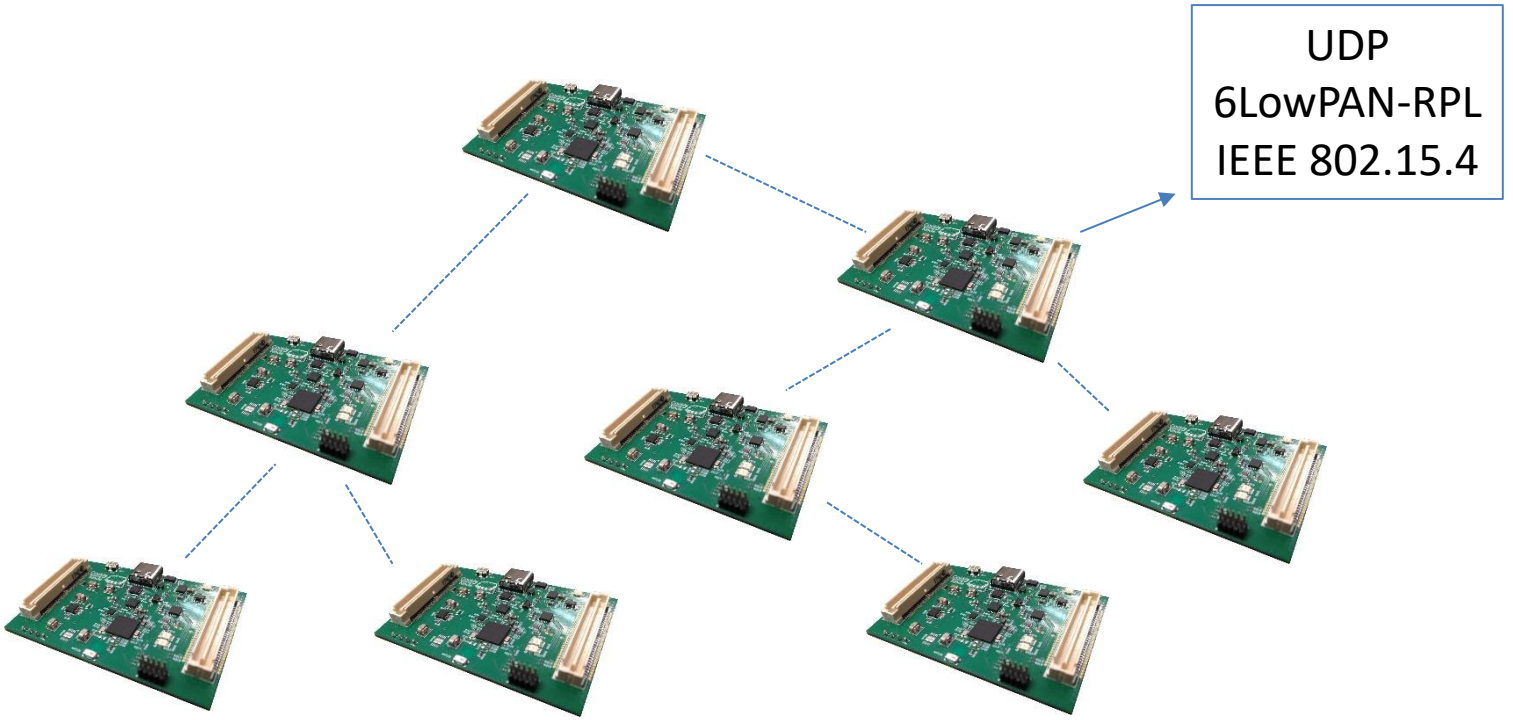
UNIVERSIDAD POLITÉCNICA DE MADRID

cei@upm.es

POLITÉCNICA

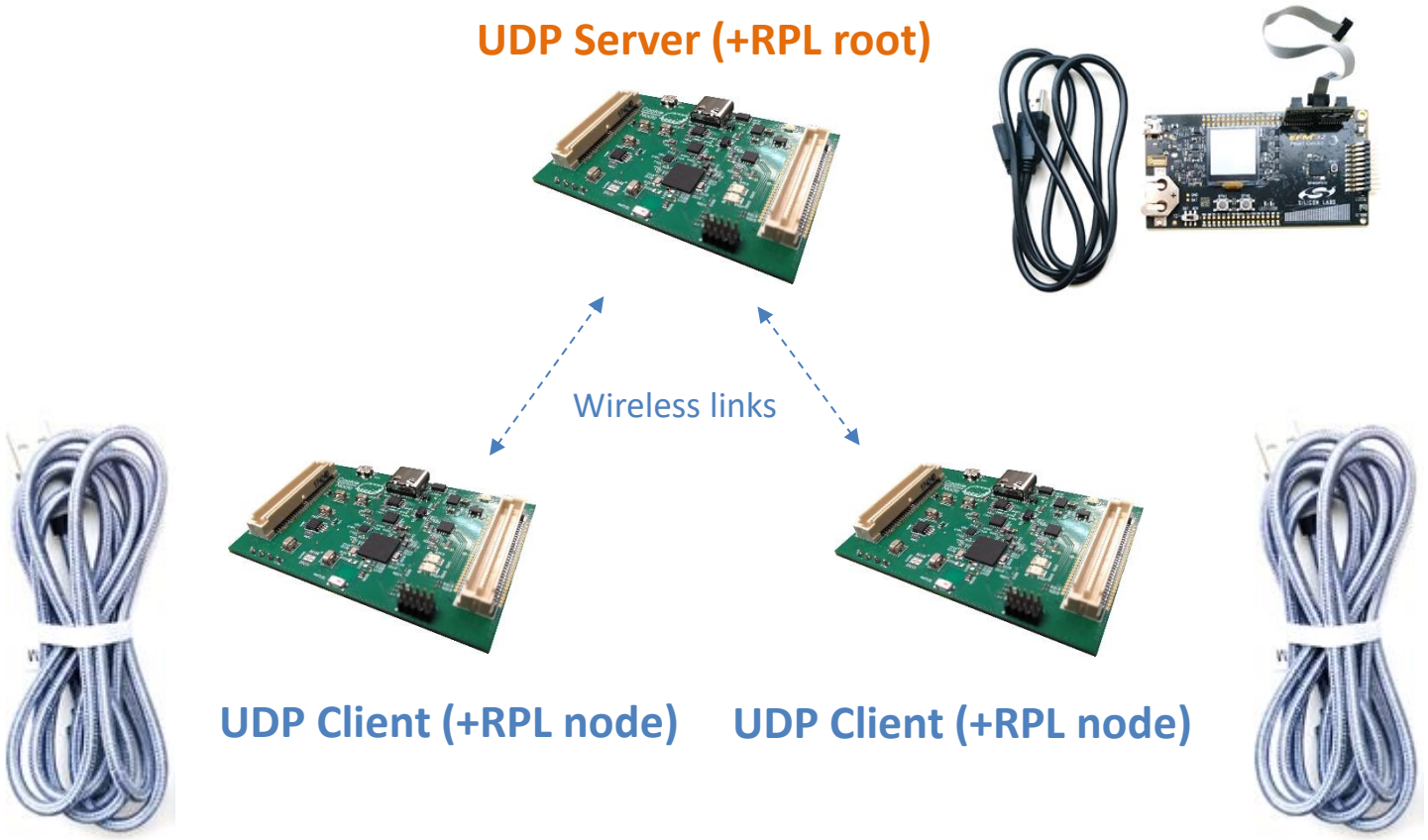
Contiki-NG – Wireless Communication

- Use of the radio communication module of the EFR32 SoC in combination with the Networking Stack of Contiki-NG, to create Wireless Sensor Networks.



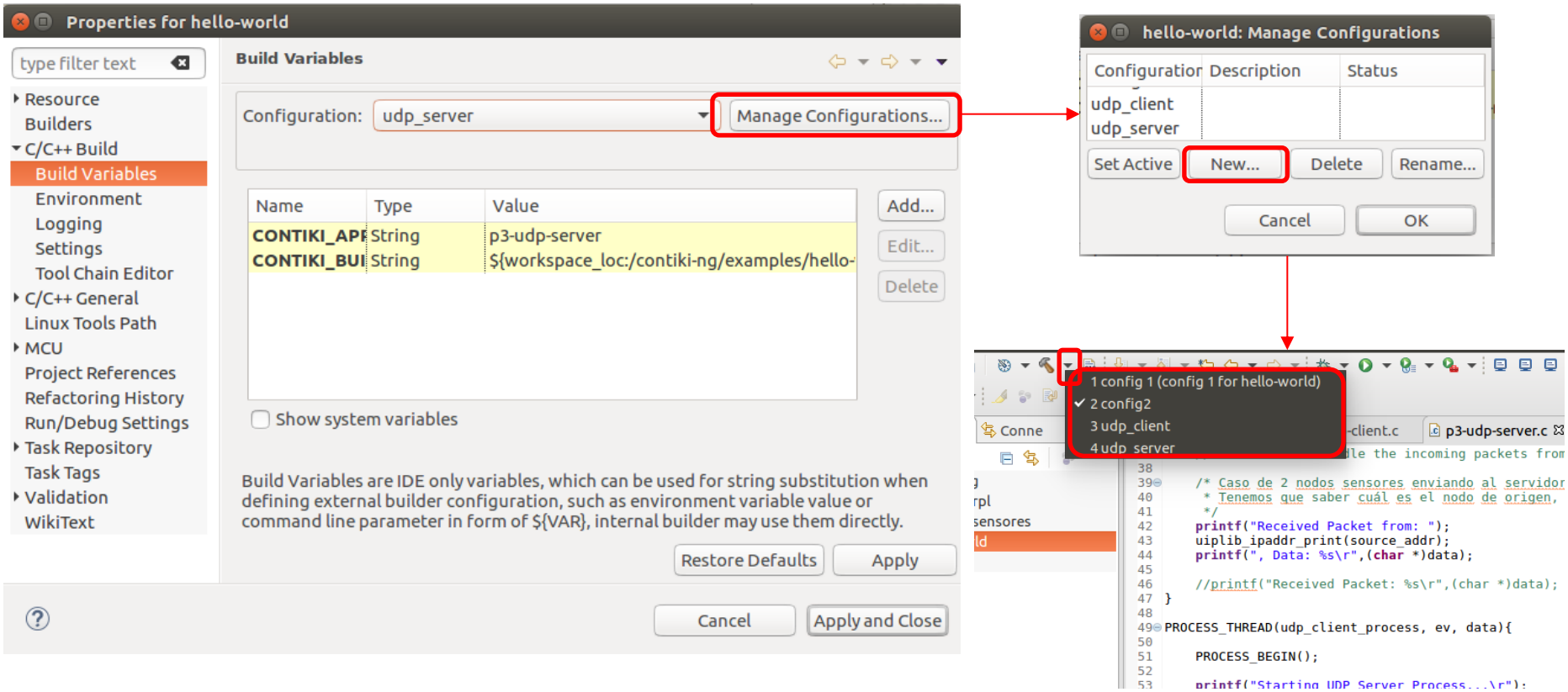
Contiki-NG – Wireless Communication

- Creation of a UDP Server-Client communication over 6LowPAN
- We will create **individual PANs** for communication between Server/Client nodes



Contiki-NG – Wireless Communication

- Creation of different building configurations in Eclipse to easily switch from one building to another (useful for agile Server/Client compilation and upload process).



Contiki-NG – Wireless Communication

- Modification of the **PAN ID** to uniquely identify each network (the channel can also be changed by the configuration macros):

```
34 #ifndef CONTIKI_H_
35 #define CONTIKI_H_
36
37 #include "contiki-conf.h"
38 #include "contiki-default-conf.h"
39
40 #include "sys/process.h"
41 #include "sys/autostart.h"
42
43 #include "sys/timer.h"
44 #include "sys/ctimer.h"
45 #include "sys/etimer.h"
46 #include "sys/rtimer.h"
47
48 #include "sys/pt.h"
49
50 #include "sys/clock.h"
```

contiki.h → contiki-default-conf.h

```
31
32 #ifndef CONTIKI_DEFAULT_CONF_H
33 #define CONTIKI_DEFAULT_CONF_H
34
35 /*---------------------------------------------------------------------------*/
36 /* Link-layer options
37  */
38
39 /* IEEE802154_CONF_PANID defines the default PAN ID for IEEE 802.15.4 networks */
40 #ifndef IEEE802154_CONF_PANID
41 #define IEEE802154_CONF_PANID 0xabcd
42 #endif /* IEEE802154_CONF_PANID */
43
44 /* IEEE802154_CONF_DEFAULT_CHANNEL defines the default channel for IEEE 802.15.4
45  * networks, for MAC layers without a channel selection or channel hopping
46  * mechanism. Current 802.15.4 MAC layers:
47  * - CSMA: uses IEEE802154_CONF_DEFAULT_CHANNEL
48  * - TSCH: uses its own TSCH_DEFAULT_HOPPING_SEQUENCE instead
49  */
50 #ifndef IEEE802154_CONF_DEFAULT_CHANNEL
51 #define IEEE802154_CONF_DEFAULT_CHANNEL 26
52 #endif /* IEEE802154_CONF_DEF_CHANNEL */
```

PANID

Number of channel (2,4 GHz)

Contiki-NG – Wireless Communication

- Creation of a *udp-server.c* source file (for example, within the hello-world project):

```
#include "contiki.h"
#include <stdio.h>
#include "net/routing/routing.h"
#include "net/netstack.h"
#include "net/ipv6/simple-udp.h"
#include "net/ipv6/uiplib.h"

/** REMEMBER: First change the PANID in contiki-default-conf.h:
 * CLick on the include "contiki.h", then click on the include
 * contiki-default-config.h, and modify IEEE802154_CONF_PANID
 */

#define UDP_CLIENT_PORT 8765
#define UDP_SERVER_PORT 5678

#define MY_MESSAGE "Test UDP Server"
static struct simple_udp_connection udp_conn;
```

Libraries for RPL multi-hop and wireless communication

Definition of the UDP Socket ports

Message to be sent

UDP connection struct

Creation of the RPL Tree (Root)

```
PROCESS_THREAD(udp_server_process, ev, data){

    PROCESS_BEGIN();

    printf("Starting UDP Server Process...\r");

    NETSTACK_ROUTING.root_start();

    simple_udp_register(&udp_conn, UDP_SERVER_PORT,NULL,
                        UDP_CLIENT_PORT,udp_rx_callback);

    PROCESS_END();
}
```

UDP Socket register

```
void udp_rx_callback(struct simple_udp_connection *c,
                     const uip_ipaddr_t *source_addr,
                     uint16_t source_port,
                     const uip_ipaddr_t *dest_addr,
                     uint16_t dest_port,
                     const uint8_t *data, uint16_t datalen){

    // Here we will handle the incoming packets from UDP connection:

    printf("Received Packet from: ");
    uiplib_ipaddr_print(source_addr);
    printf(", Data: %s\r",(char *)data);

}
```

Implementation of the UDP RX Callback function to handle incoming packets

Contiki-NG – Wireless Communication

- Creation of a *udp-client.c* source file (for example, within the hello-world project).
- Use of the same functionalities as in case of the Server, that is:
  - UDP Socket port definition
  - UDP Socket Register
  - RX Callback implementation for handling the UDP socket incoming packets
- In case of the UDP Client, the node will join the RPL tree created by the Root node (the server in this case).
- The client gets the IP address of the server and then sends periodic data to it.

UDP Socket register

Periodic UDP packet transmission

```
static struct etimer et;
uip_ipaddr_t dest_address;

PROCESS_BEGIN();

printf("Starting UDP Client Process...\r");

simple_udp_register(&udp_conn, UDP_CLIENT_PORT,NULL, UDP_SERVER_PORT,udp_rx_callback);

etimer_set(&et, CLOCK_SECOND*2);

while(1){
    PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&et));

    if(NETSTACK_ROUTING.node_is_reachable() && NETSTACK_ROUTING.get_root_ipaddr(&dest_address)){
        simple_udp_sendto(&udp_conn, MY_MESSAGE, sizeof(MY_MESSAGE), &dest_address);
    }
    else{
        printf("Node not Reachable yet...\r");
    }

    etimer_reset(&et);
}


PROCESS_END();
```

Contiki-NG – Wireless Communication

- Exercises to be carried out:
  - Send periodic data from the client to the server and then the server answers a reply packet back to the client.
  - Send periodic sensing data from the client to the server.
  - The server shows in a terminal the sensor measurements received from the client.